

Artificial Intelligence

Def:

Artificial → "Human made".

Intelligence → "Thinking power".

It is a branch of computer science by which we can create intelligent machines which can behave like a human, think like humans, and able to make decision.

Example : ⇒ Self-driving car

⇒ playing chess

⇒ Google Assistant

⇒ Painting

Why we use Artificial Intelligence?

1. AI machines can solve real-world problems easily and accurately. (Ex: Traffic Issue, Health Issue)
2. Virtual Assistant.
3. With AI we can build robots
4. opens a path for other new technology.

Goals of AI

- ①. Replicate Human intelligence
2. Solve knowledge intensive tasks.
3. An intelligent connection of perception and action
4. Building a machine which can perform tasks that requires human intelligence. such as
 - playing chess
 - Surgical operation
 - Driving a car in traffic
5. Creating some system which can exhibit intelligent behavior; learn new things by itself and can advice to its user.

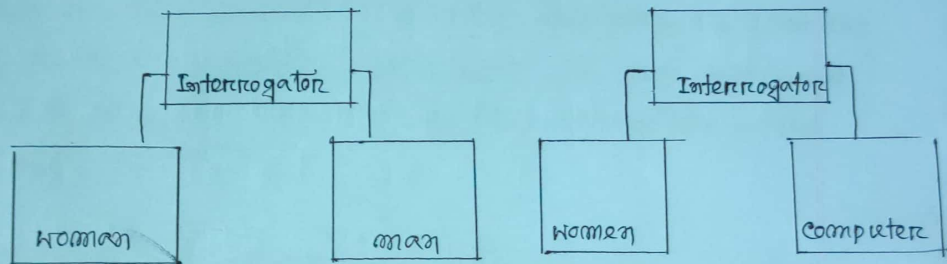
Application of AI

1. AI in Healthcare
2. Gaming
3. Social Media
4. Travel and transport
5. Robotics
6. Entertainment
7. Agriculture
8. Education.

Can machine think?

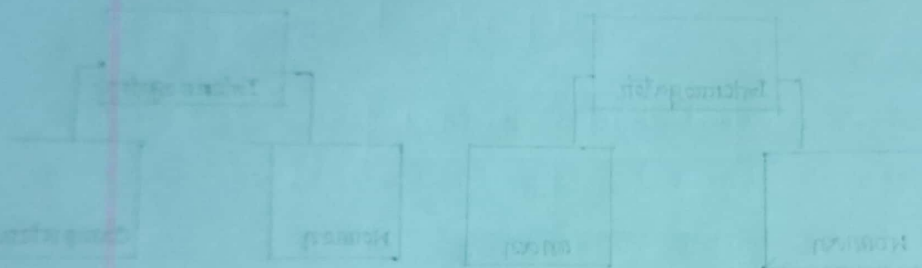
Yes / No

Turing Test.



Intelligence is composed of following things

1. Linguistics Intelligence
2. Problem Solving
3. Perception
4. Learning
5. Reasoning.



Problem Solving: Problem solving is commonly known as the method to reach the desired goal or finding a solution to a given situation.

Problem Solving in AI: The user can solve the problem by performing logical algorithms, utilising polynomial and differential equation and executing them using modeling paradigms.

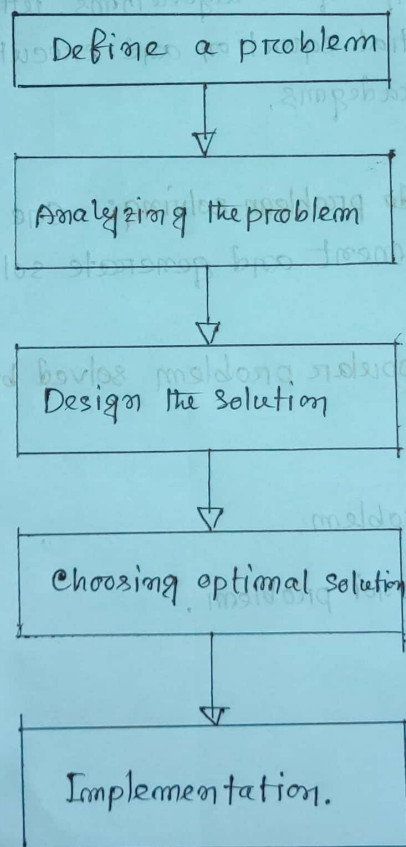
To do this / To do problem solving: One needs to define problem statement and generate solution.

Some most popular problems solved by AI

1. chess
2. N-Queen Problem
3. Tower of Hanoi problem.

Problem Searching: Problem searching is the most commonly used technique of problem solving in AI. Finding Information one need.

The process of Solving problem:



Properties of searching Algorithm:

1. **Completeness:** A searching algorithm is said to be complete if it is guaranteed to return a solution if at least one solution exists for any random input.
2. **Optimality:** Best solution (lowest-path-cost)
3. **Time complexity:** Measure of time for an algorithm to complete its task.
4. **Space Complexity:** How much memory is needed to perform the task.

Types of searching algorithm

1. Uninformed / Brute Force Method / Blinding search
2. Informed search / Heuristic search.

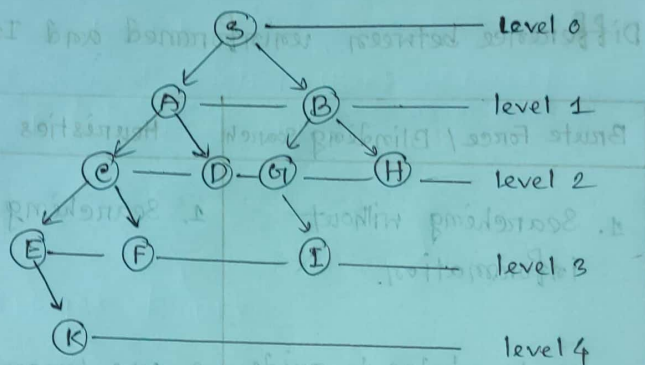
Difference between uninformed and Informed search

Brute Force / Blinding Search	Heuristics Search
1. Searching without information.	1. Searching with information
2. No knowledge to guide	2. use knowledge to find steps to solution.
3. Time consuming	3. Quick solution
4. More complexity (Time / space)	4. less complexity (Time / space).
5. Ex: BFS, DFS	5. Ex: A* Best First Search.

uninformed / Brute Force / Blinding Search.

" Breath First Searching (BFS)."

Ex Root node (S) and Goal node (K).



S
A B
B C D
D G H
D G H E F
G H E F
H E F I
E F I
F I K
I K
K

→ uninformed

→ FIFO (Queue)

→ Shallowest Node

→ Complete

→ optimal

$O(b^d)$ → Time complexity

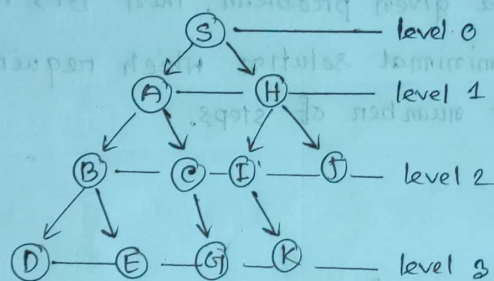
$$T(b) = 1 + b^1 + b^2 + \dots + b^d$$

Path → S → A → B → C → D → G → H → E → F → I → K.

uninformed search.

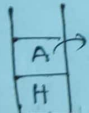
"Depth First searching (DFS)".

Ex Root node (S) to Goal node (G).

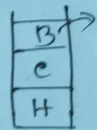


Path $\rightarrow S \rightarrow A \rightarrow B \rightarrow D \rightarrow E \rightarrow G$

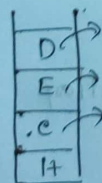
$\rightarrow$ uninformed



$\rightarrow$ LIFO (stack)



$\rightarrow$ Deepest Node



$\rightarrow$ Incomplete

$\rightarrow$ Non-optimal



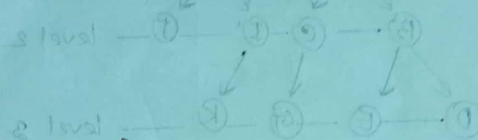
$\rightarrow$ Time complexity.

$$T(b) = 1 + b^1 + b^2 + \dots + b^d = O(b^d).$$

$$\text{space complexity} = O(bm).$$

Advantages of BFS

1. BFS will provide a solution if any solution exists
2. If there are more than one solution for a given problem, then BFS will provide the minimal solution which required the least number of steps.



Disadvantages of BFS

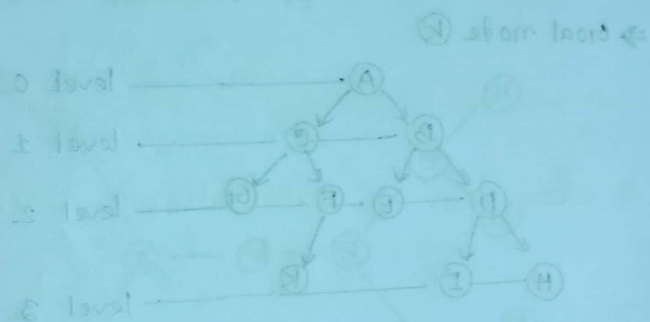
1. required lots of memory space
 $O(b^d)$.
2. required lots of time for solution.
 $O(b^d)$.

Disadvantages of DFS

1. No guarantee to find the solution
2. It may go the infinite loop.

Difference between BFS and DFS

BFS → Breadth First Search
DFS → Depth First Search



Iteration 1: A
Iteration 2: B, C
Iteration 3: D, E, F, G
Iteration 4: H, I, J, K, L, M, N, O

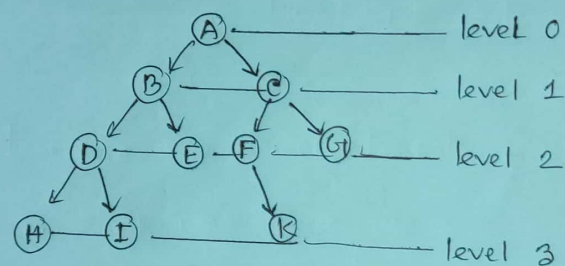
Time complexity = $O(V)$
Space complexity = $O(V)$

Uninformed search (Iterative Deepening Search) IDS
Combination of BFS and DFS algorithm.

BFS $\rightarrow$ fast search

DFS $\rightarrow$ Memory Efficiency.

$\Rightarrow$ Goal node (K)



একভাবে প্রথমে DFS কাজ করে এবং পরবর্তীতে BFS কাজ করে
অর্থাৎ প্রথমে deep এর সমস্ত স্থান।

Iteration 1 : A

Iteration 2 : A, B, C

Iteration 3 : A, B, D, E, F, G

Iteration 4 : A, B, D, H, I, E, C, F, K, G,

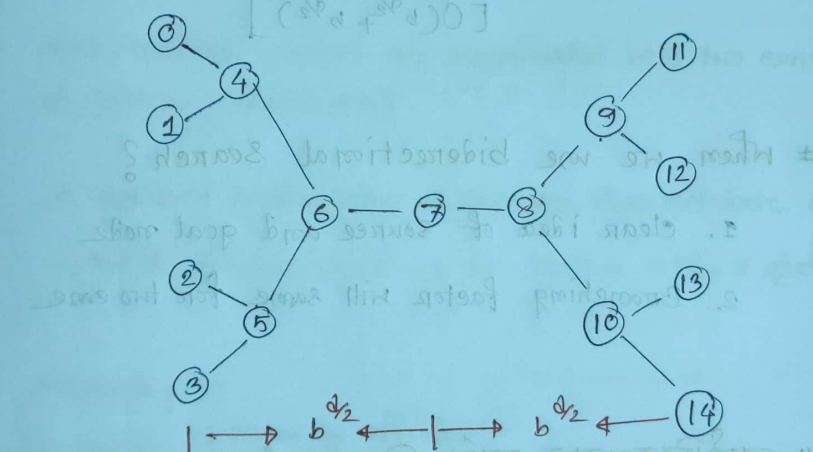
Time Complexity = $O(b^d)$

space complexity = $O(bd)$

Bidirectional Search

1. Forward Search : $S \rightarrow \text{Goal}$ (source to goal)
2. Backward Search : $G \rightarrow \text{Source}$ (goal to source)

$\Rightarrow$ Source = 0 and Goal = 14



For BFS and DFS

Time complexity = $O(b^d)$ where b = branching factor
 d = depth factor

Why we use bidirectional search?

For Bidirectional search

Space/ Time complexity = $O(b^{d/2} + b^{d/2})$.

$$O(b^d) > O(b^{d/2} + b^{d/2}).$$

For reducing complexity we use bidirectional search.

Performance.

- Complete
- Optimality
- Time and space complexity

$$[O(b^{\frac{b}{2}} + b^{\frac{b}{2}})]$$

When we use bidirectional search?

1. Clear idea of source and goal node
2. Branching factor will same for two case

વાઈડિકેશન આર્ચ વિશ્લેષણ (BFS) કવશર કરવું

Depth limited search algorithm (DLS)

DLS is the solution of DFS with a predetermined limit. DLS can solve the drawback of the infinite path in DFS.

Depth-limited search can be terminated with two conditions of failure (limitation)

1. Standard Failure Value $\rightarrow$ Problem does not have any solution
2. Cut off failure value $\rightarrow$ No solution within a given depth limit.

Advantages:

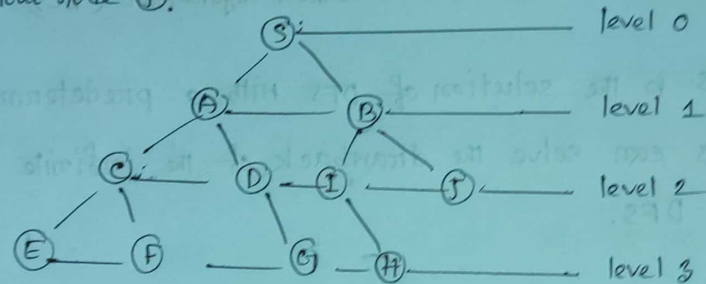
$\rightarrow$ Memory efficient

Disadvantages:

$\rightarrow$ Incompleteness

$\rightarrow$ It may not be optimal. If the problem has more than one solution.

⇒ Goal node (J)



limit → level-2

Path → S → A → C → D → B → I → J

Time complexity $O(b^l)$

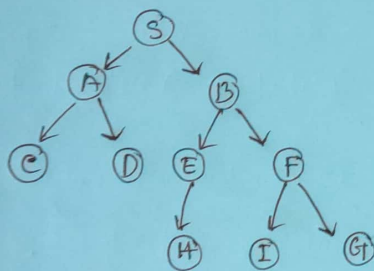
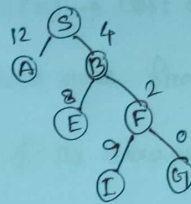
Space complexity $O(bl)$

$l < d \rightarrow$ incomplete

$l > d \rightarrow$ optimal

Informed Search

"Best First Search"



straight line distance

$$S \rightarrow G = 13$$

$$A \rightarrow G = 12$$

$$B \rightarrow G = 4$$

$$C \rightarrow G = 7$$

$$D \rightarrow G = 3$$

$$E \rightarrow G = 8$$

$$F \rightarrow G = 2$$

$$H \rightarrow G = 4$$

$$I \rightarrow G = 9$$

$$G \rightarrow G = 0$$

Array

n	h(n)
S	13
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	4
I	9

① uses only heuristic function

$h(n)$ = cost from n to goal

→ Implemented by Priority Queue

→ Select path best at that moment

→ use BFS and DFS

→ Time and space complexity $O(b^d)$.

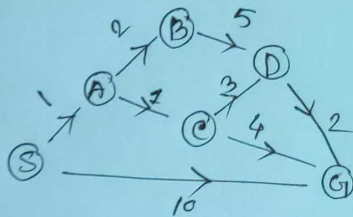
→ incomplete and non-optimal

Informed search (A^* search).

$f(n) = g(n) + h(n) \rightarrow$ Cost of reach node n to goal node

$\downarrow$
cost of reach node from start node

Estimated cost of the cheapest solution.



n	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Initialization $f(s) = g(s) + h(s)$
 $= 0 + 5 = 5$

Iteration 1: $S \rightarrow A$ $S \rightarrow G$
 $1 + 3 = 4$ $10 + 0 = 10$

Iteration 2: $S \rightarrow A \rightarrow B$ $S \rightarrow A \rightarrow C$ $S \rightarrow G$
 $1 + 2 + 4 = 7$ $1 + 1 + 2 = 4$ $10 + 0 = 10$

Iteration 3: $S \rightarrow A \rightarrow C \rightarrow G$ $S \rightarrow A \rightarrow C \rightarrow D$ $S \rightarrow G$
 $1 + 1 + 4 + 0 = 6$ $1 + 1 + 3 + 6 = 11$ $10 + 0 = 10$

A* Admissibility

Node	h
A	2
B	1
C	0
D	3
E	1
F	0



$$f(n) = g(n) + h(n)$$

$$f = g + h$$

$$1 \leq 2 \quad A \leftarrow B \quad \therefore \text{not allowed}$$

$$0 = 0 + 0 \quad F \leftarrow C$$

$$2 \leq 2 \quad D \leftarrow A \quad \therefore \text{not allowed}$$

$$1 + 0 = 1 \quad E \leftarrow B \quad \therefore \text{not allowed}$$

$$2 \leq 2 \quad F \leftarrow C \quad \therefore \text{not allowed}$$

$$1 + 0 = 1 \quad F \leftarrow C \quad \therefore \text{not allowed}$$